Faculdade de Engenharia da Universidade do Porto



Acme Electronic Supermarket: Smart Shopping and Payment System

'MEIC - Computação Móvel'

Group 04

João Pedro Antão Sobral (up202108736) Participation: 30%

João Pedro Moreira Costa (up202108714) Participation: 30%

Rafael Neves Teixeira (up202108831) Participation: 40%

Index

1. Summary **3**
2. Introduction **3**
3. System Overview **3**
4. User Registration and Authentication **6**
5. Shopping Process **7**
6. Checkout Procedure **7**
7. Payment Processing and Gate Control **8**
8. Transaction History and Voucher Retrieval **8**
9. System Architecture **9**
10. Server API Endpoints **9**
User Management 9
Voucher Management 10
Checkout and Payment 10
Product and Shopping Basket 10
Administrative 10
11. User Interface and Experience **11**
12. Admin Authentication and Interface **11**
13. Data Schemas **12**
Database Tables 12
14. Performed Tests **13**
Functional Tests 13
15. External Libraries Used 15
16. How to Run the Application **15**
1. Set the Network Configuration 15
2. Start the Backend Server 15
3. Access the SQLite Database (via WSL) 15
4. Admin Credentials 16
17. Conclusion **16**
18. References 17
19. Appendix **18**
UI App Flows 18

1. Summary

This project was developed in the '*Computação Móvel*' UC context and the main goal was to ensure the implementation of a supermarket application to allow customers to purchase their products without requiring the store's personnel to scan their products.

2. Introduction

The Acme Electronic Supermarket is set to transform the traditional retail model by introducing a fully cashierless shopping and payment system. This innovative approach leverages RFID technology and an Android-based mobile application to streamline the customer experience, improve efficiency, and enhance security. Customers will be able to scan products using their smartphones as they shop, manage their virtual basket in real time, and complete purchases through secure, self-service checkout terminals. By integrating real-time data synchronization, cryptographic security measures, and a seamless user interface, the system aims to deliver a fast, convenient, and modern alternative to conventional supermarket checkout processes.

3. System Overview

The system comprises three main components:

1. **Customer Android Application**: Enables users to register, scan products, manage their shopping basket, view transaction history, and apply loyalty vouchers.
2. **Checkout Terminal Application**: Installed on Android devices at store exits, it communicates with the customer app to process payments and control gate access.
3. **Remote Server (HTTP REST Service)**: Handles customer registration, transaction processing, voucher management, and maintains the central database.

Figures 1 and 2 illustrate the system's workflow, providing a clearer representation of its operation.

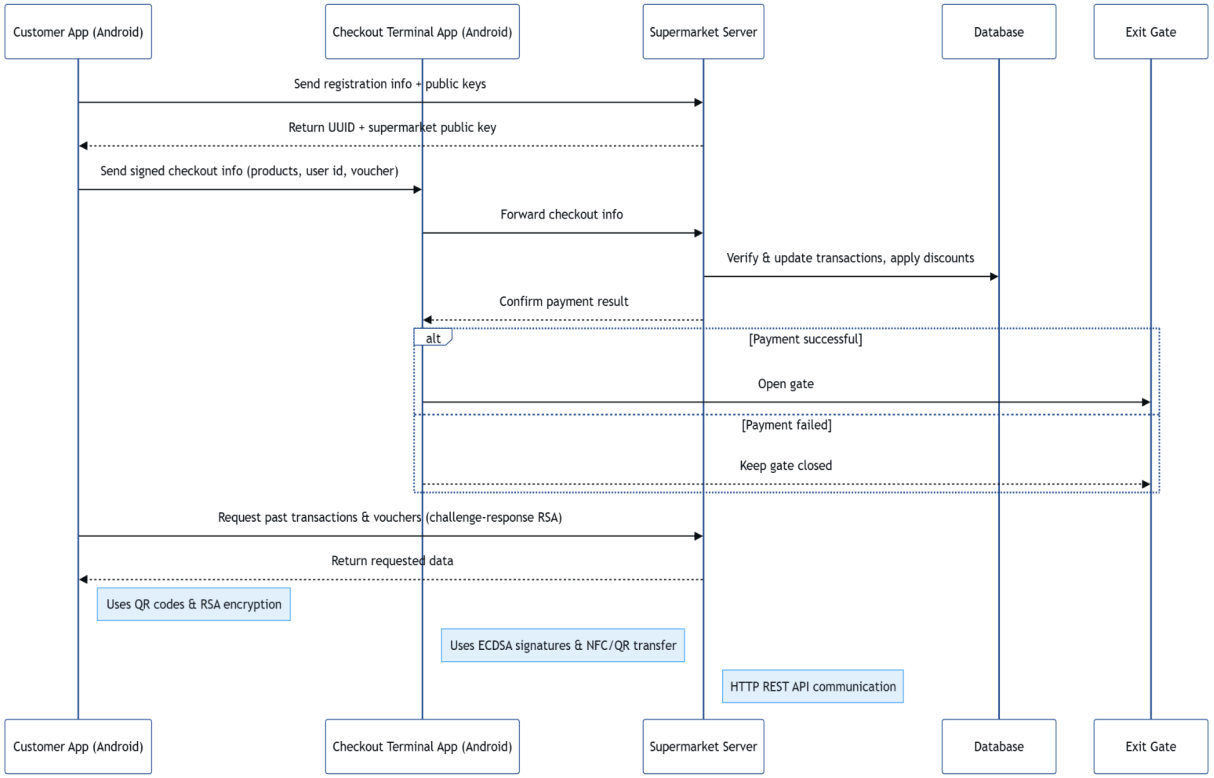


Fig. 1 - Sequence diagram representing the flow of the system

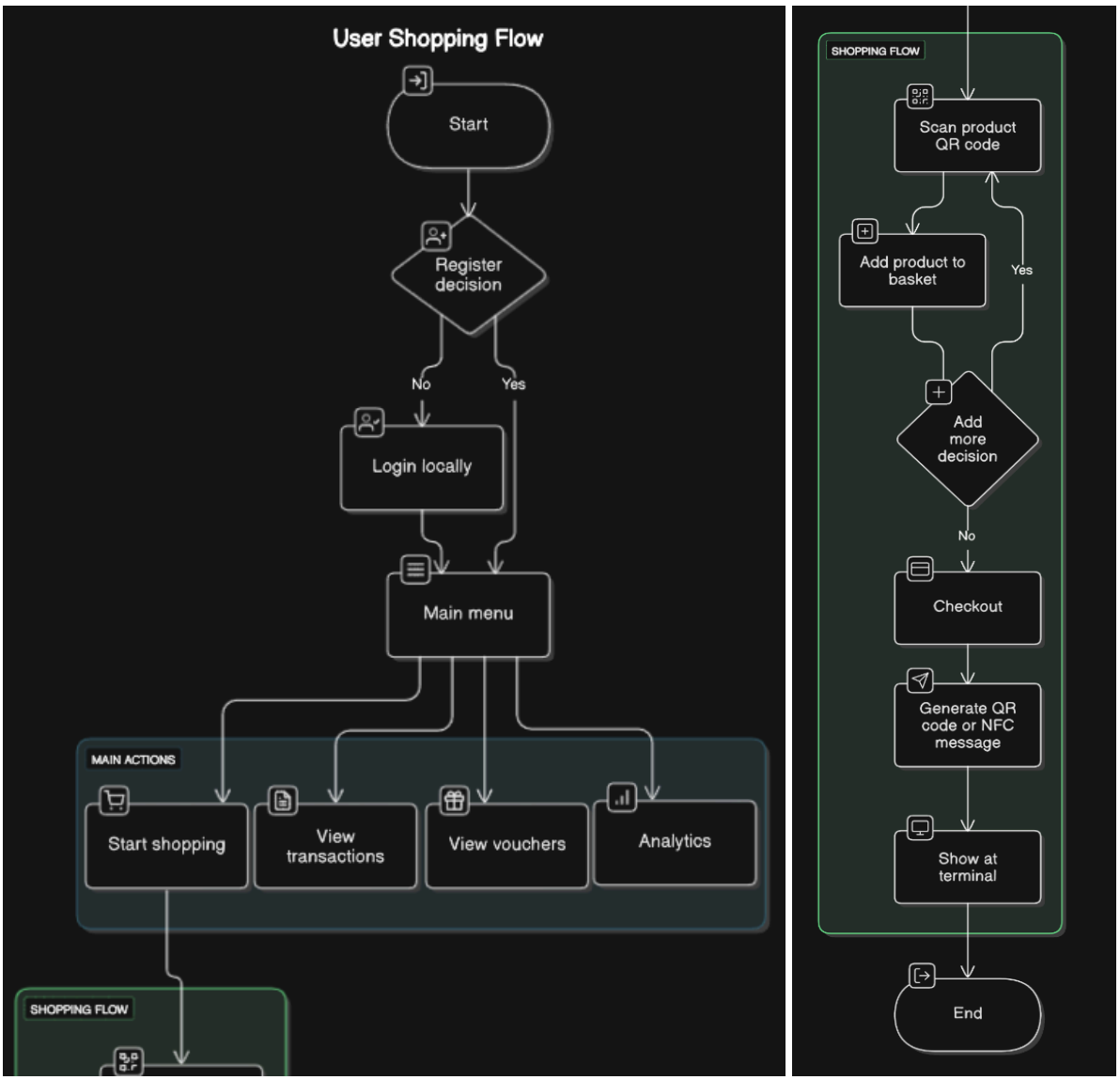


Figure.2 - Flow Chart demonstrating how a client interacts with the system

4. User Registration and Authentication

Upon installing the app, users must register by providing personal information and payment details. The app generates two asymmetric cryptographic key pairs (RSA with 512 bits encrypted with **RSA/NONE/PKCS1Padding** and EC), storing the private keys securely on the device. The public keys are sent and stored in the server, in the **Users** database table with the fields PublicRSA and PublicEC. Then, the server responds with a unique user identifier (UUID). Since our server is running locally, we ended up not transmitting the supermarket's RSA public key during this request as we didn't find a good, practical and safe enough solution to simulate it. We are aware that this is not a safe thing to do and that we would not replicate this method on a real application, but, since we don't have a seperate server to run on its own environment, what we did, was just hard-code a manually generated pair of RSA keys for the supermarket in the code to simulate a Paper Wallet approach.

To enhance registration security, we implemented two additional safeguards. Firstly, upon form submission, the user's password is encrypted and securely stored locally on the device. During subsequent logins, users are prompted to enter only their username and password. The login process verifies identity by comparing the entered password with the decrypted version stored on the device. This mechanism strengthens security by preventing unauthorized access, but introduces a trade-off—accounts are limited to a single device due to the local storage of credentials.

Secondly, both registration and login incorporate a challenge-response protocol using a cryptographic nonce. When accessing the app, the server sends a nonce to the client, which is then signed using the user's private EC key. The server verifies the response using the user's public EC key, ensuring that only the legitimate user can authenticate successfully. This adds an additional layer of cryptographic identity verification to prevent impersonation and replay attacks.

We used the "androidx.security.crypto" library to encrypt the locally stored password.

5. Shopping Process

As customers shop, they scan QR codes on product labels using the app. Each QR code contains:

- A unique product identifier (UUID)
- Product name
- Price information

The QR codes are created by an admin and encrypted with the supermarket's RSA private key to prevent tampering. When an admin creates a QR code, the product that represents that QR code is added to the database table **Product**.

The app decrypts the information using the provided public key and adds the item to the user's virtual shopping basket in the server's database (**UserProduct** database table).

A client can only proceed to the checkout if he has at least 1 product in his basket.

We used the ZXing library for the QR Code scanning functionality.

6. Checkout Procedure

When ready to check out, the app compiles:

- A list of product IDs and prices
- The user's UUID
- An optional voucher ID
- A flag indicating whether to apply accumulated discounts

This information is signed using the user's EC private key, encrypted using the **SHA256withECDSA** algorithm, and presented as a QR code or NFC message. The checkout terminal reads this data and forwards it to the server for processing.

7. Payment Processing and Gate Control

When checking out, we provide the client with options. To either checkout using a QR Code or a NFC tag. After the client picks one option and scans the tag that appears on the terminal, the server verifies the signature using the user's EC public key and processes the payment:

- Calculates the total amount
- Applies any accumulated discounts if requested
- Validates and applies the voucher discount (15% of the total purchase)
- Updates the user's accumulated spending and discount records
- Issues a new voucher for every €100 spent

The server sends the transaction result back to the terminal. If successful, the terminal displays the outcome and a green screen and opens the exit gate. In case of failure, a red screen and error are shown, and the gate remains closed.

8. Transaction History and Voucher Retrieval

Users can request their transaction history and available vouchers at any time. The server authenticates the request using a challenge-response mechanism:

1. The server sends a random nonce.
2. The app signs the nonce with the user's RSA private key and returns it.
3. The server verifies the response using the stored public key.

Upon successful authentication, the server sends the requested data, which the app displays to the user.

We ended up signing the nonce and not encrypting it as requested because, from the problems that what we encountered and the research that we made, private key encryption is not supported in most modern crypto libraries like Android Keystore (the one we used), Node.s and OpenSSL. This is due to the fact that encryption using a private key exposes

structure that can be abused using, for example, padding oracle attacks. So, in order to prevent breaches in security, and to allow us to verify the user identity using the challenge-nonce, we ended up signing and not encrypting the nonce with the user's RSA private key.

9. System Architecture

The system follows a client-server architecture:

- **Client Side**: Android applications for customers and checkout terminals.
- **Server Side**: RESTful API handling business logic, database interactions, and security validations.
    - **Node.js Server:** Built using the Koa framework, it provides RESTful APIs for user, product, voucher, and payment management. It handles cryptographic verification and database operations.
    - **SQLite Database:** Stores user, product, voucher, and transaction data.

This modular design allows for scalability and easy maintenance.

10. Server API Endpoints

The server exposes a RESTful API used by both the customer app and the checkout terminal app to handle user authentication, voucher management, product tracking, and payment processing. Below is a breakdown of the main endpoints and their responsibilities:

User Management

- `GET /users`: Retrieves all registered users (admin access).
- `GET /users/:id`: Fetches data for a specific user by ID.
- `POST /users`: Registers a new user in the system, including cryptographic public keys and personal/payment info.
- `PUT /users/:id`: Updates the information of an existing user.

- `DELETE /users/:id`: Deletes a user from the system.
- `POST /login_user`: Handles user login by verifying the locally stored encrypted password.
- `POST /get_nonce`: Provides a cryptographic nonce for general challenge-response authentication during login.
- `POST /get_voucher_nonce`: Issues a nonce specifically for voucher retrieval authentication.
- `POST /analytics/:id`: Requests the user analytics

Voucher Management

- `POST /vouchers`: Creates a new voucher (issued by the server when user spending crosses thresholds).
- `GET /vouchers`: Retrieves all available vouchers in the system.
- `POST /vouchers/:id`: Retrieves vouchers associated with a specific user (using POST to submit the signed nonce along with the request)
- `GET /vouchersUsers`: Returns all the connections between users and their respective vouchers.

Checkout and Payment

- `POST /checkout`: Finalizes a purchase. Verifies the signed checkout data, validates final price, applies discounts, generates new vouchers (if the case), processes payment and returns the result to the terminal.

Product and Shopping Basket

- `POST /products`: Adds a new product to the system (admin access).
- `POST /addShopList`: Adds a scanned product to a user's current shopping basket.
- `GET /getShopList/:id`: Retrieves the current shopping basket for a given user.
- `POST /userProduct/:id`: Deletes a product from the user's basket.

Administrative

- `POST /admin`: Verifies if a user has administrative privileges.

11. User Interface and Experience

The Android app features an intuitive interface:

- **Registration**: Simple form for entering personal and payment information.
- **Shopping**: Real-time basket updates as products are scanned.
- **Checkout**: Clear prompts for applying discounts and vouchers.
- **History and Vouchers**: Accessible records of past transactions and available vouchers.
- **Client Analytics:** Page that displays the client's purchase analytics like total amount spent in the store, number of purchases, average amount spent, total discount, number of vouchers used and the top 5 products bought.

Screenshots and detailed UI flows are included in the appendix.

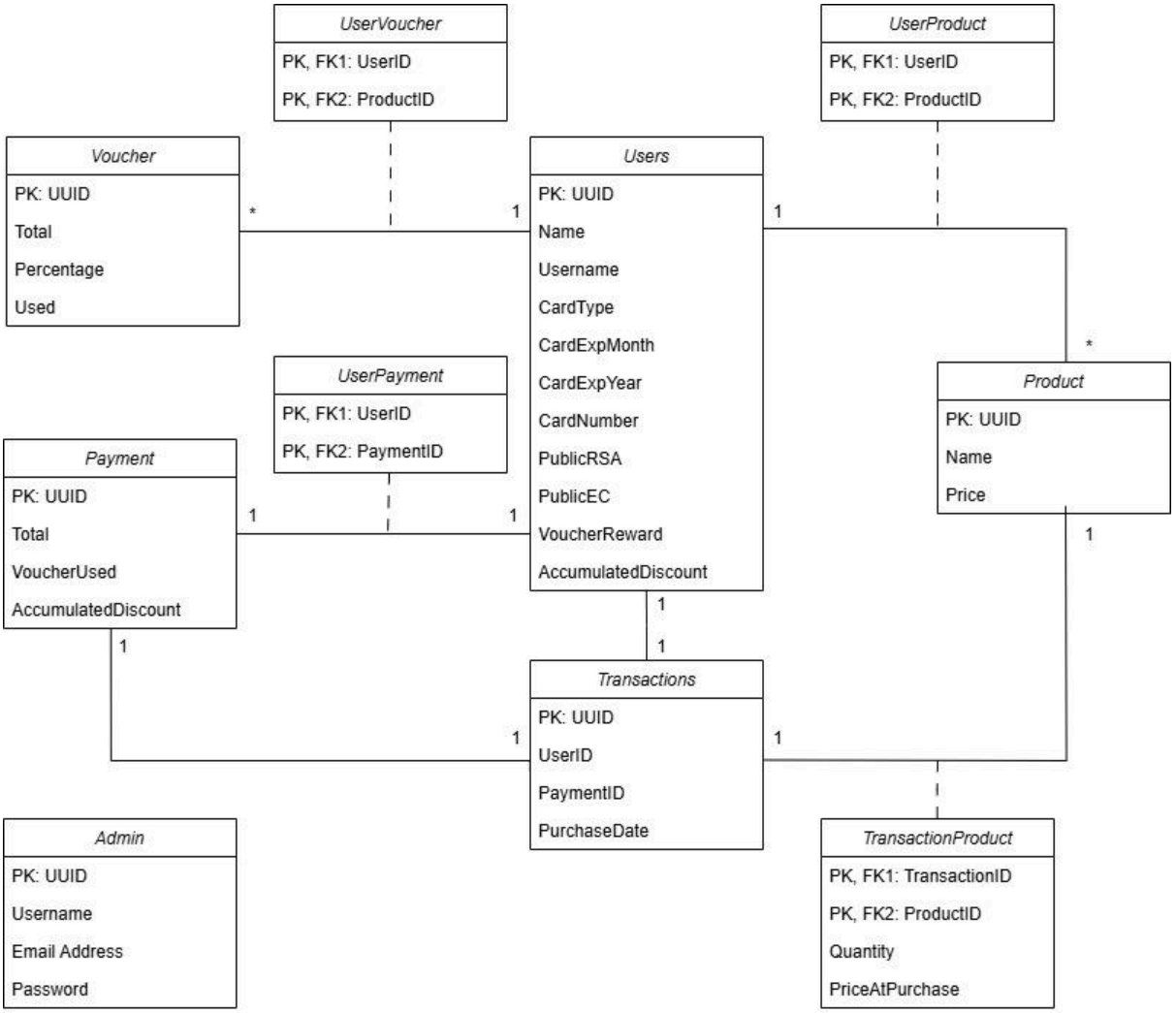
12. Admin Authentication and Interface

An admin authentication system was implemented, distinct from the regular user authentication. This divergence was intentional, as we aimed to experiment with alternative authentication mechanisms. Unlike the user-side approach, which relies on public and private keys used locally, the admin login uses a more traditional method, a **server-side password authentication**. The admin password is hashed and securely stored on the server, which is then used to verify an admin's login.

Additionally, an admin interface was developed to support two main functionalities: adding vouchers to users and generating QR codes for store products.

- **Add Vouchers**: The admin can assign vouchers to users by specifying the user's username, the voucher name and the discount percentage.
- **Generate Product QR Codes**: By clicking the *Generate* button, the admin can generate a random UUID for a product, input the product name and price (entered in euros and cents), and finalize the process by clicking *Generate Tag*. This action adds the product to the database.
- **Scan Shopping List**: To test the terminal tag scan.

13. Data Schemas



Database Tables

The database schema is defined in database.sql. Key tables include:

Users

- **Columns:** UUID, Name, Username, CardType, CardExpMonth, CardExpYear, CardNumber, PublicRSA, PublicEC, VoucherReward, AccumulatedDiscount.
- **Purpose:** Stores user information, including cryptographic keys for secure communication.

Product

- **Columns:** UUID, Name, Price.
- **Purpose:** Stores product details.

Voucher

- **Columns:** UUID, Name, Percentage, Used.
- **Purpose:** Tracks discount vouchers.

Payment

- **Columns:** UUID, Total.
- **Purpose:** Records payment transactions.

Transactions

- **Columns:** UUID, UserID, PaymentID, PurchaseDate.
- **Purpose:** Records each purchase event made by a user

Relationships

- **UserProduct:** Links users to products.
- **UserVoucher:** Links users to vouchers.
- **UserPayment:** Links users to payments.
- **TransactionProduct:** Link transaction to products.

14. Performed Tests

Functional Tests

User Registration:

- Verified that users can register with valid data.
- Ensured RSA and EC keys are securely stored and transmitted.
- Simulated a man-in-the-middle attack to tamper with the public key exchange during registration. However, since the public key is securely stored on the user's device and never transmitted again after registration, unauthorized access to the user's account would require physical access to the device and its secure storage making such an attack infeasible.

Product Management:

- Tested adding, updating, and deleting products via the server API.
- Verified product details are correctly retrieved by the client.

Voucher Handling:

- Checked voucher generation based on user spending.
- Tested applying vouchers during checkout.
- Tried using someone else's key to encrypt the nonce received by the server during the challenge-nonce. The server, when verifying the signed nonce, rejected the request.
- If we try to use someone else's voucher UUID or reuse a spent voucher, the server rejects as it verifies if the voucher belongs to the client or if it has already been used.

Checkout Process:

- Verified encryption and signing of checkout data.
- Ensured the server correctly decrypts and validates the data.
- Manually re-signed a tampered payload with a different EC key pair not belonging to the user. The server rejected it.
- Attempted to reuse a previously signed checkout QR code but, since we generate a new nonce every time a checkout request is made, replay attacks are prevented.

Transaction Tests:

- Tried using someone else's key to encrypt the nonce. The server, when verifying the signed nonce, rejected it.
- Attempted to reuse a previously signed transaction request but, since we generate a new nonce every time a transaction request is made, replay attacks are prevented.

Security Tests:

- Tested nonce expiration and invalid signatures.
- Verified secure storage of keys and sensitive data.

QR Code Tamper:

- Tried forging a QR-Code with fake product info. Since our QR Codes are encrypted with the supermarket's private key, safety is ensured.

Nonce Replay in Challenge-Response

- Attempted to reuse an old nonce during voucher/transaction retrieval but since nonces are deleted after every transaction and new ones are generated when making a request, nonce-replay attacks are prevented.

15. External Libraries Used

- bcryt; koa; ZXing; crypto-js; androidx.security.crypto

16. How to Run the Application

To run the application, you'll need a **Windows OS** with **SQLite3** installed.

1. Set the Network Configuration

- If you're using the **Android emulator**, ensure the `BASE_ADDRESS` macro is set to `10.0.2.2`. This macro is inside “app/src/main/java/acme_supermarketapp/utils.kt”.
- If you're using a **physical Android device**, run `ipconfig` in your terminal to find your local IP address (typically starts with `192.x.x.x`). Set the `BASE_ADDRESS` macro to this IP to allow your device to communicate with the server.

2. Start the Backend Server

- From the root of the project:

```
cd nodeserver
node server.js
```

3. Access the SQLite Database (via WSL)

To interact with the database:

- If you're already inside the `nodeserver` folder:

```
cd database
```

- If you're in the root project folder:

```
cd nodeserver/database
```

Then, to open the SQLite terminal:

```
sqlite3 database.db
```

If the `database.db` file does not exist yet, initialize it by running:

```
sqlite3 database.db < database.sql
```

4. Admin Credentials

- username: adminOne
- pass: adminpass1

Note: If you reinstall the app and have already created an account before, you'll need to go to the app settings and press "Clean Data" since you'll need to reset the stored keys.

17. Conclusion

Developed within the *Computação Móvel* course, the Acme Electronic Supermarket system demonstrates how mobile computing and cryptographic techniques can be effectively applied to modernize retail shopping. By integrating RFID, QR codes, and secure Android-based interactions, the system enables a fully autonomous and secure customer experience, reducing dependency on traditional checkout systems.

The use of asymmetric cryptography, digital signatures, and nonce-based authentication ensured secure data exchange and user verification, while a digital voucher system introduced a practical implementation of customer loyalty features. These elements allowed us to explore real-world challenges related to secure mobile development, server communication, and usability.

This project became really interesting and beneficial to us as we gained hands-on experience in designing secure mobile systems, implementing cryptographic protocols, managing client-server architectures, and considering user experience within a real-world application. It provided valuable insight into the complexities of mobile app development and the importance of balancing functionality, security, and usability in practical deployments.

18. References

1. Classes Slides

2. Android Studio Documentation.

Android Developers. https://developer.android.com/reference/packages

Official reference for Android APIs used in development.

3. “Cryptographic Nonce.”

Wikipedia. https://en.wikipedia.org/wiki/Cryptographic_nonce

Overview of nonces and their role in secure communication protocols.

4. bcrypt Documentation.

npmjs.com. https://www.npmjs.com/package/bcrypt

JavaScript library for hashing passwords with strong cryptographic protection.

5. Koa.js Framework Documentation.

Koa.js. https://koajs.com/

Web framework for building RESTful APIs, used for server-side development.

6. ZXing Android Embedded.

JourneyApps GitHub Repository. https://github.com/journeyapps/zxing-android-embedded

Barcode scanning library used for reading QR codes in Android apps.

7. crypto-js Documentation.

npmjs.com. https://www.npmjs.com/package/crypto-js

JavaScript cryptography library offering standard algorithms like AES, SHA256.

8. Cooijmans, T., de Ruiter, J., & Poll, E. (2014).

Analysis of Secure Key Storage Solutions on Android. ACM Digital Library.
https://dl.acm.org/doi/10.1145/2666620.2666627

Academic paper discussing the security of Android key storage mechanisms.

19. Appendix

UI App Flows

When we enter the app, this is the first page that appears:

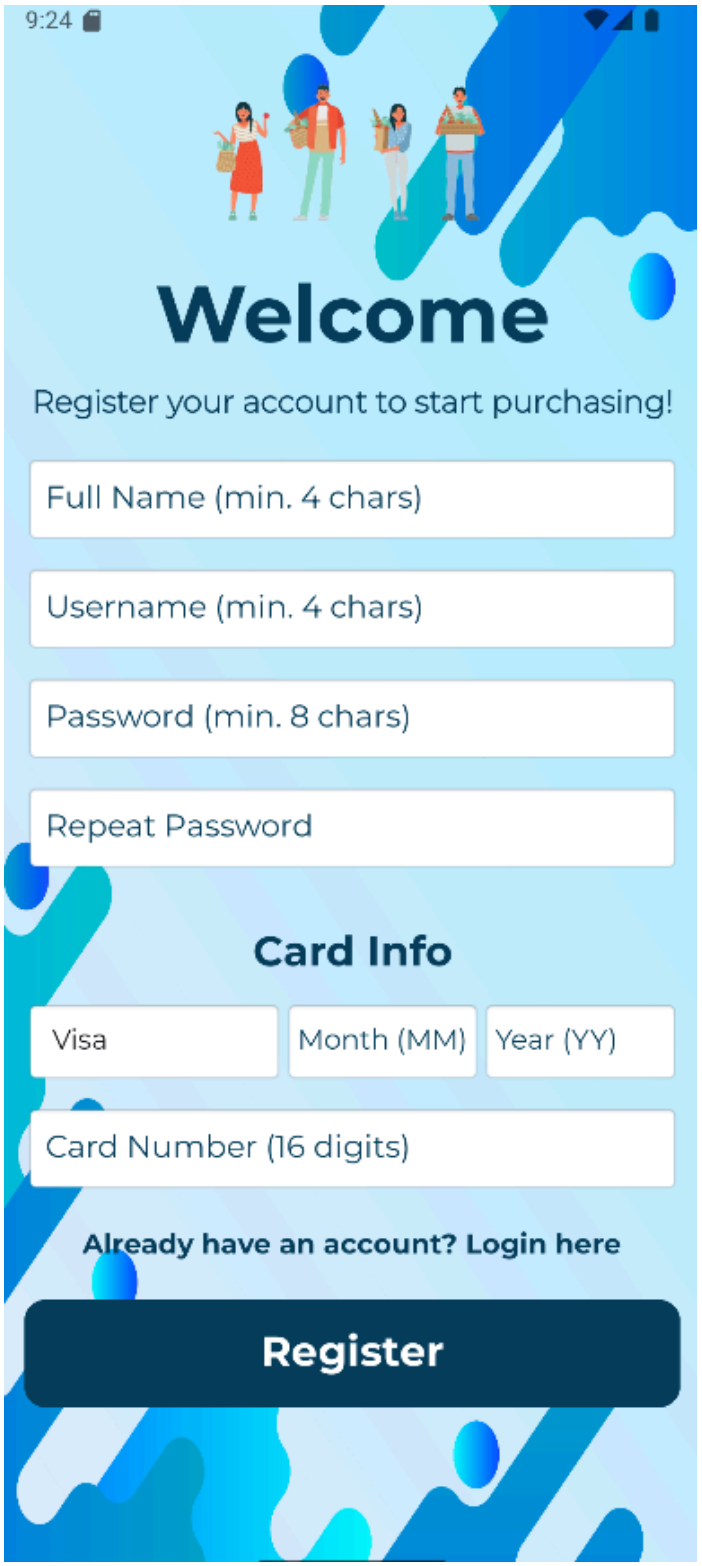


Welcome to our Supermarket!

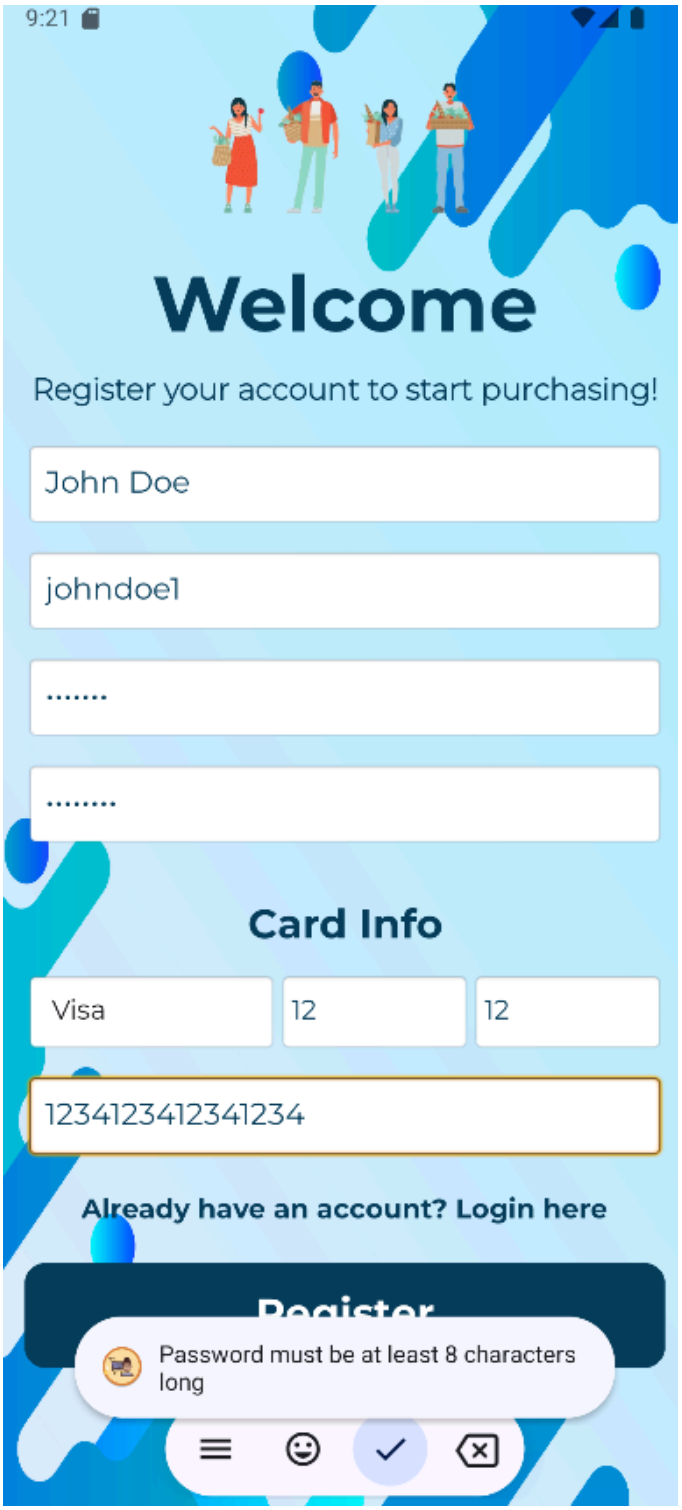
Make your purchases without any effort!

Get Started

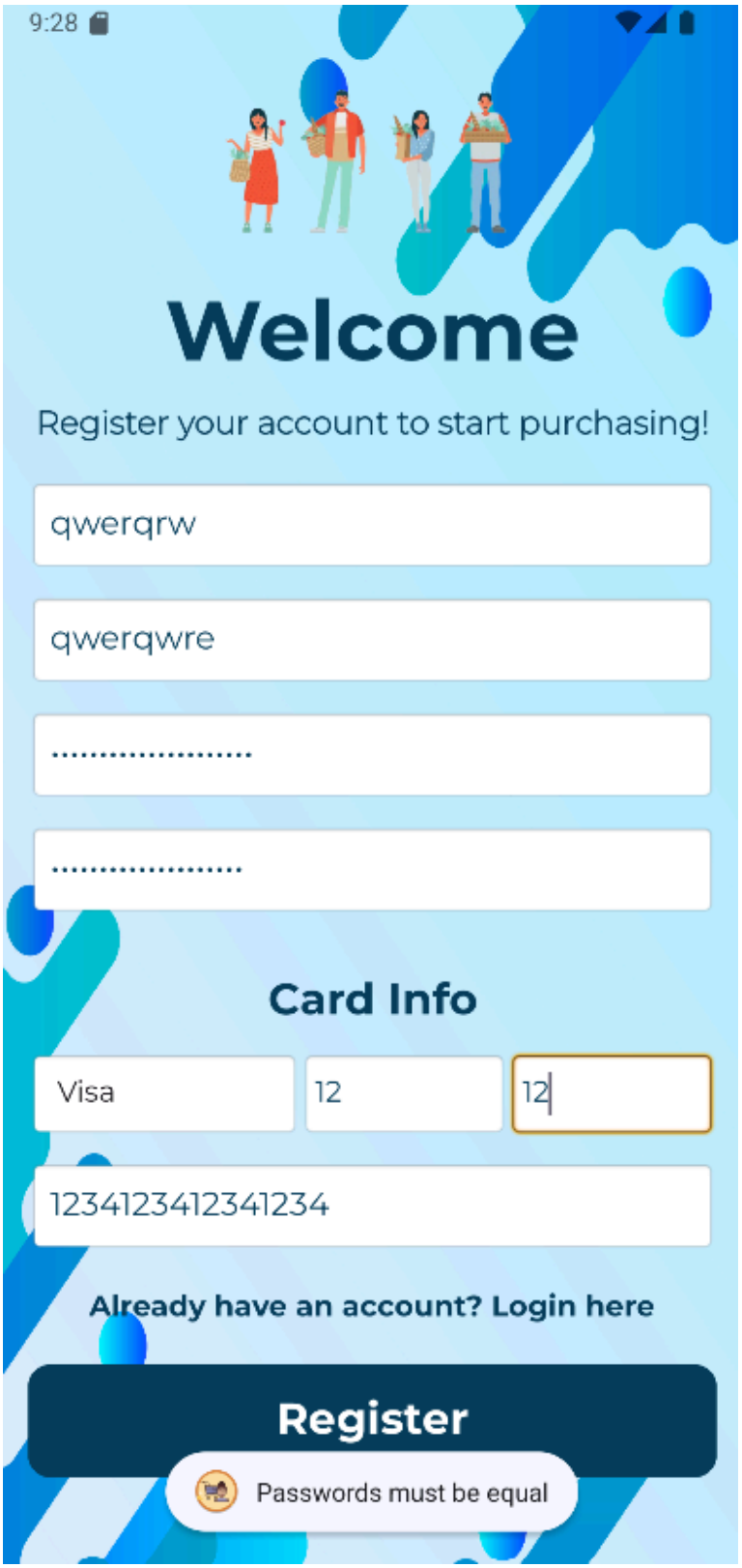
When Clicking “Get Started”, if not having an account, we are directed to the Registration Page. Otherwise we go to the Login page:



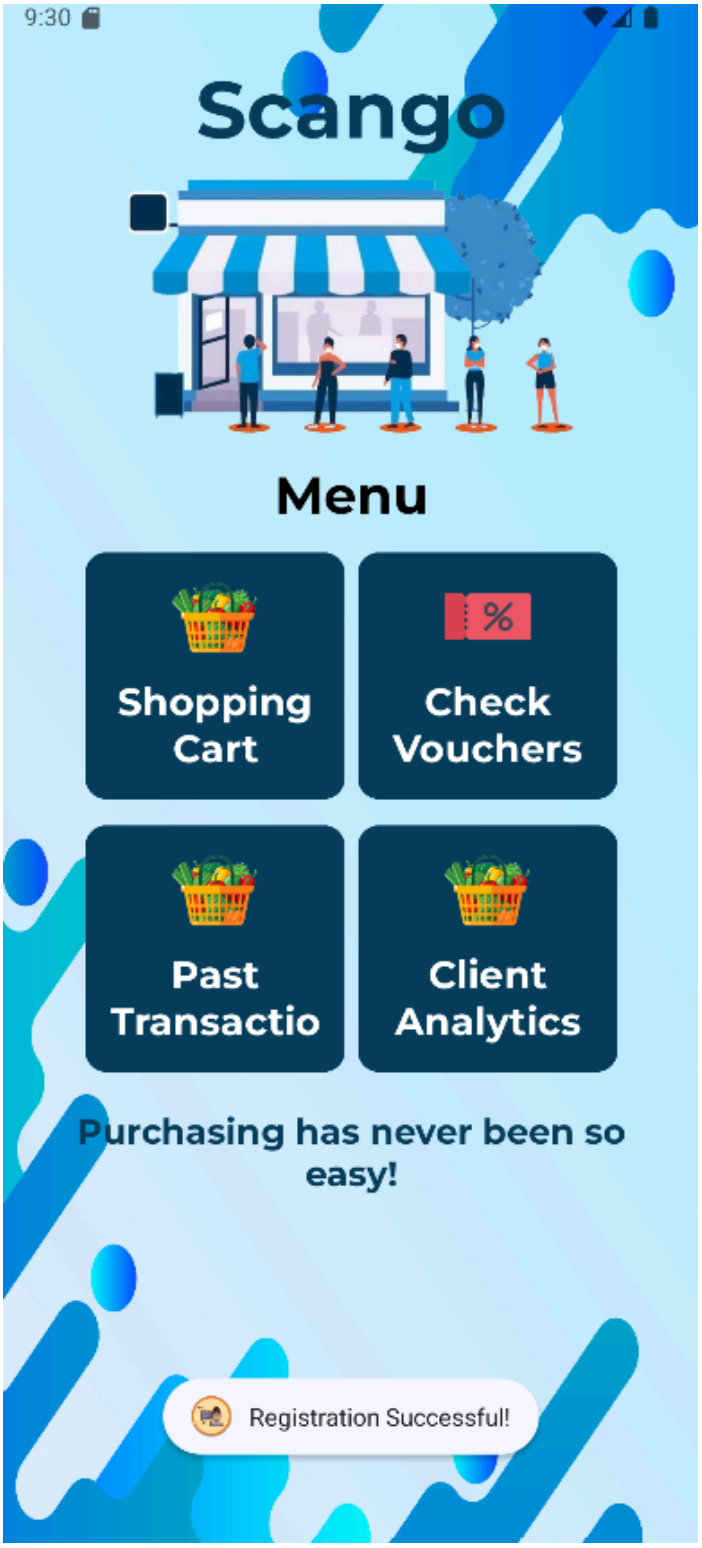
If we don’t fill the minimum number of characters on each field, we get this type of error:



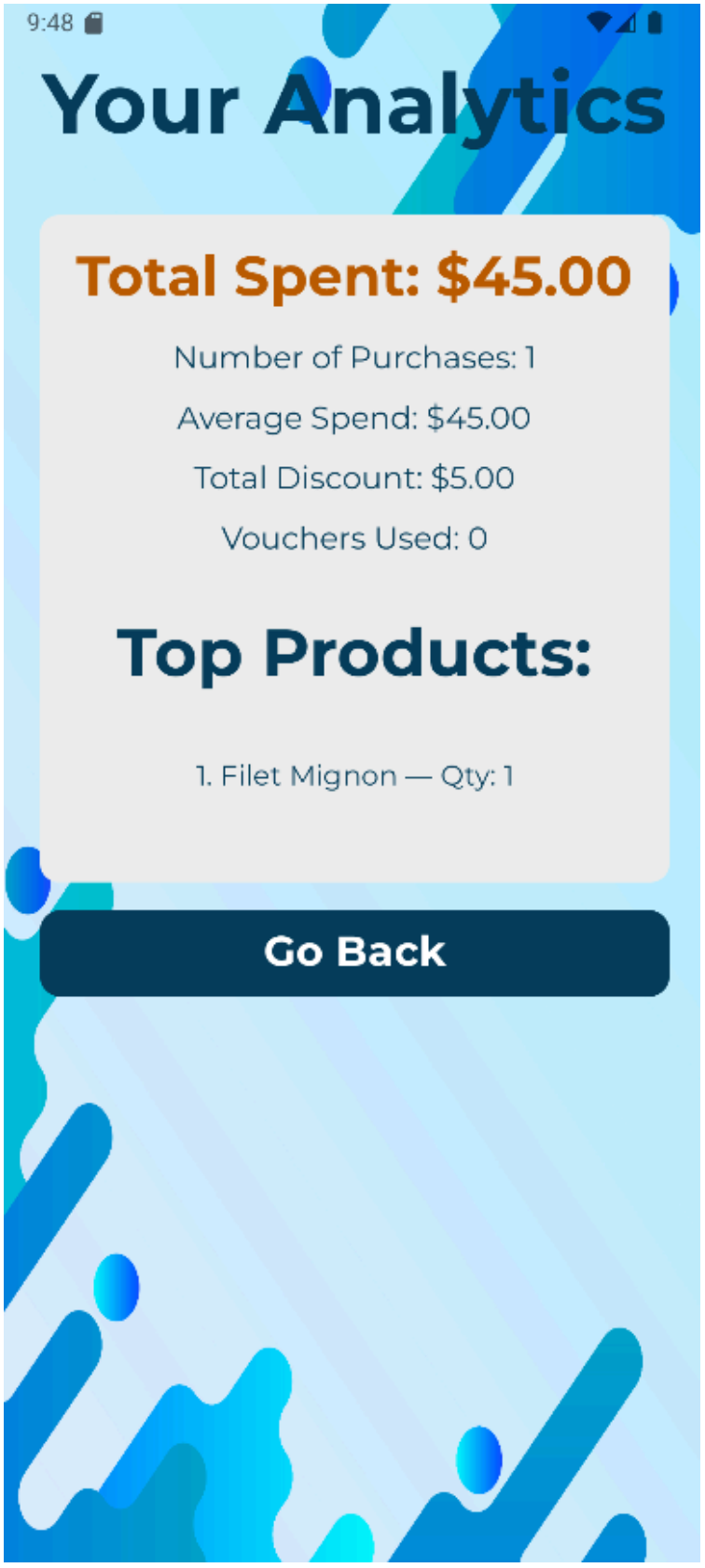
If the passwords don’t match, we get this error:



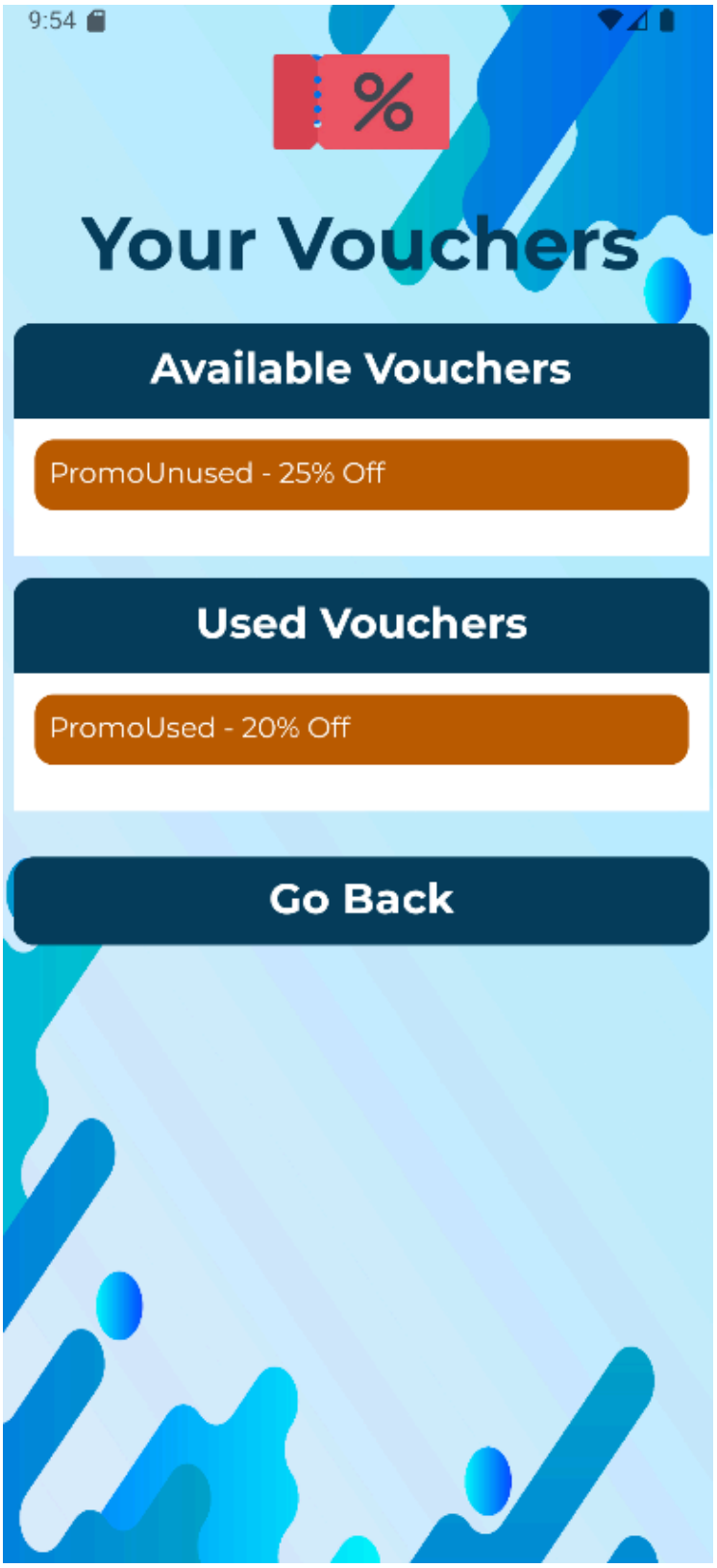
If all fields are filled correctly, and the picked username doesn't exist already in the server database, we are directed to the Home Page:



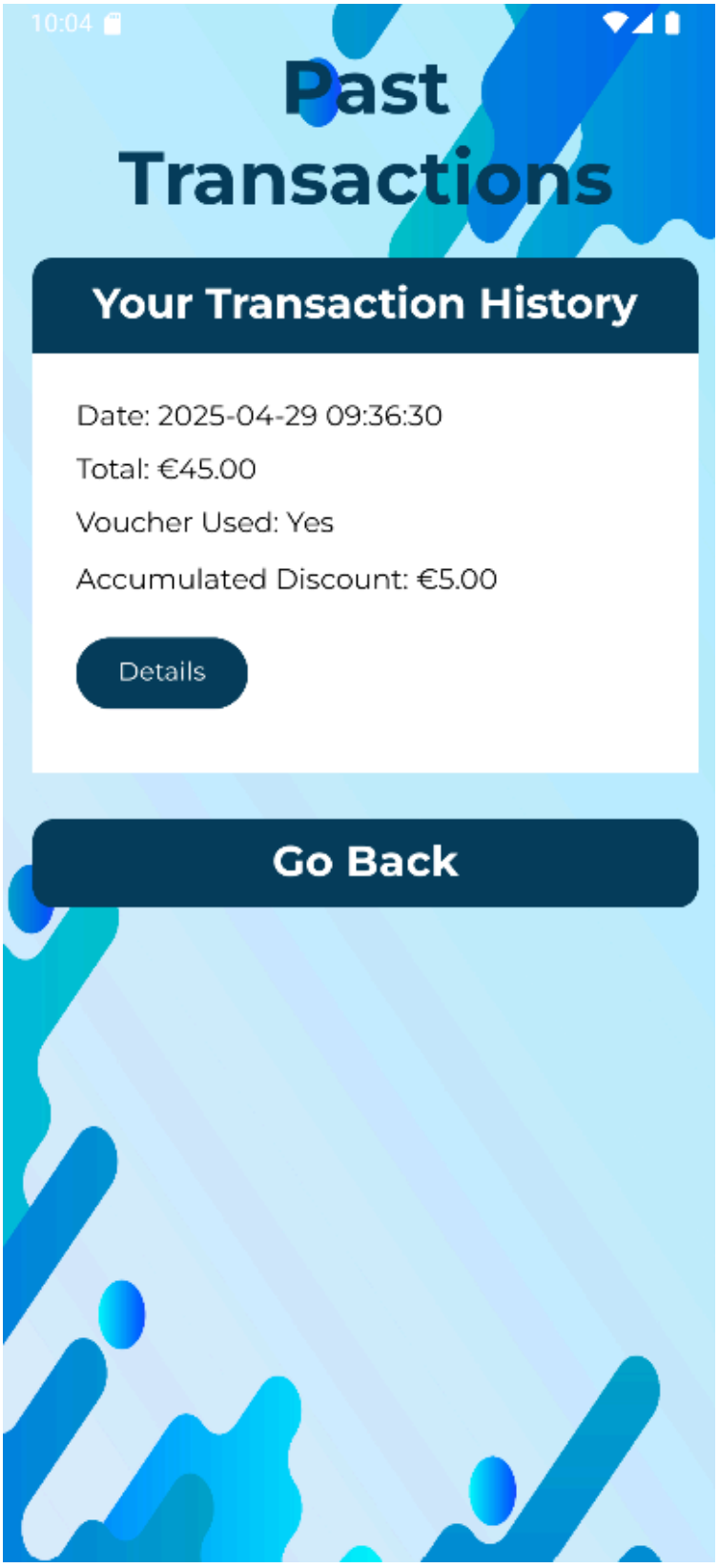
When clicking the “Client Analytics” option, we can check the statistics related to the client’s purchase:



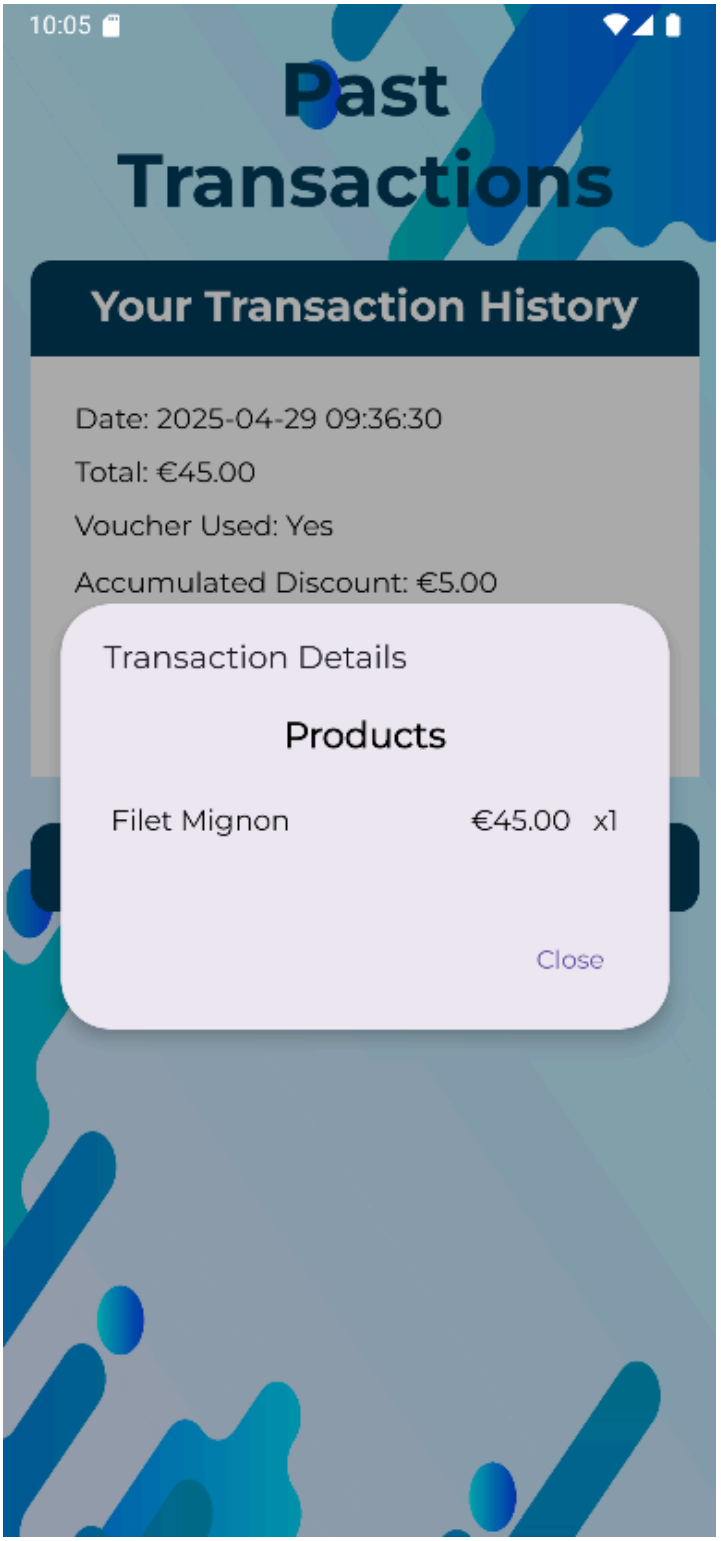
When going back to the menu, and selecting the “Check Vouchers” option, we can see the available and already used vouchers:



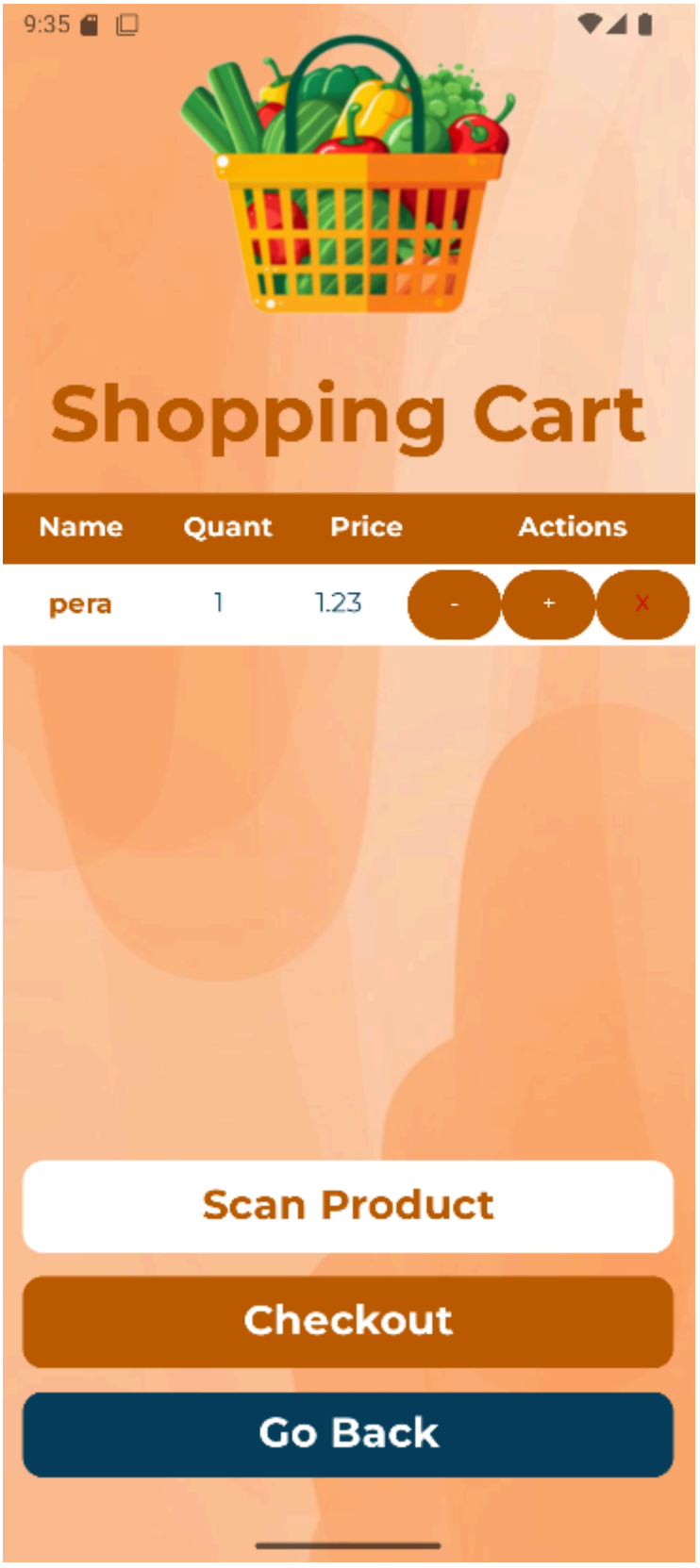
When going back to the Home Page and clicking on the “Past Transactions” option, we can see all the past transactions made in the supermarket:



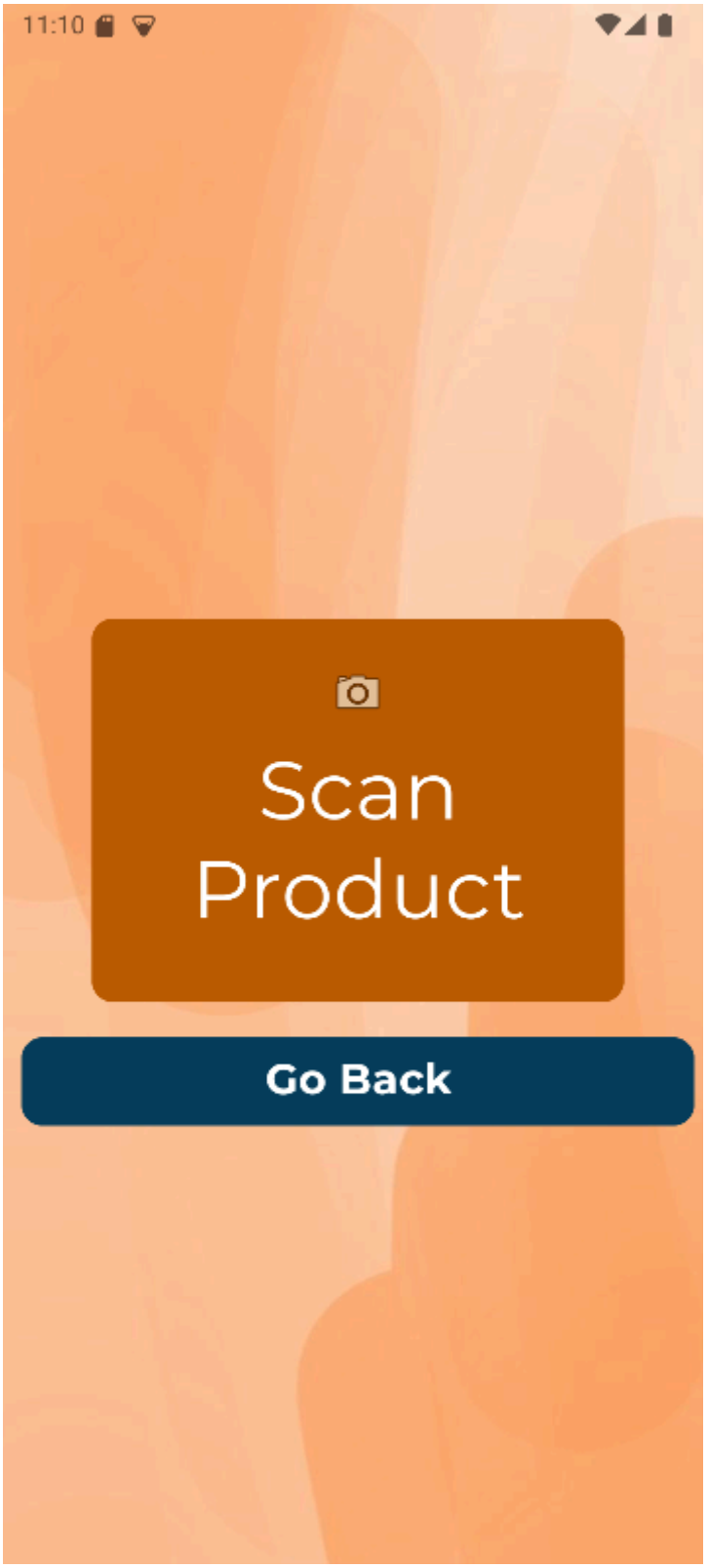
If we click on the “Details” option, we can see which products were bought and their purchase price at that time:



When going back to the Home Page, and clicking on the “Shopping Cart” option, we can see the current basket of scanned products and the option to scan products:

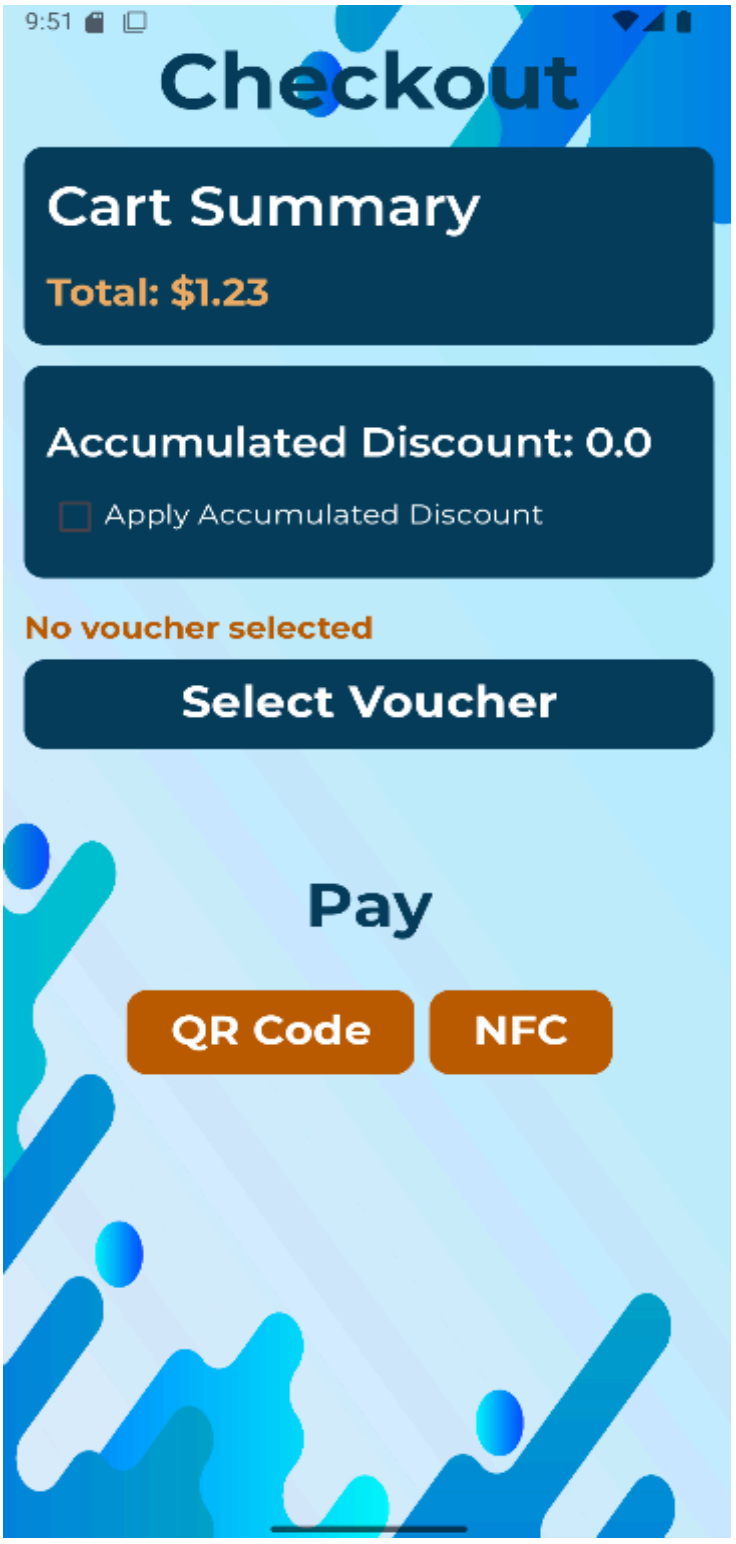


If we click on the option “Scan Product” we go to the Scan Product Page:



If we click on “Scan Product” the phone camera opens to start scanning the product. When scanning a product’s QR Code, it is added to the customer’s basket:

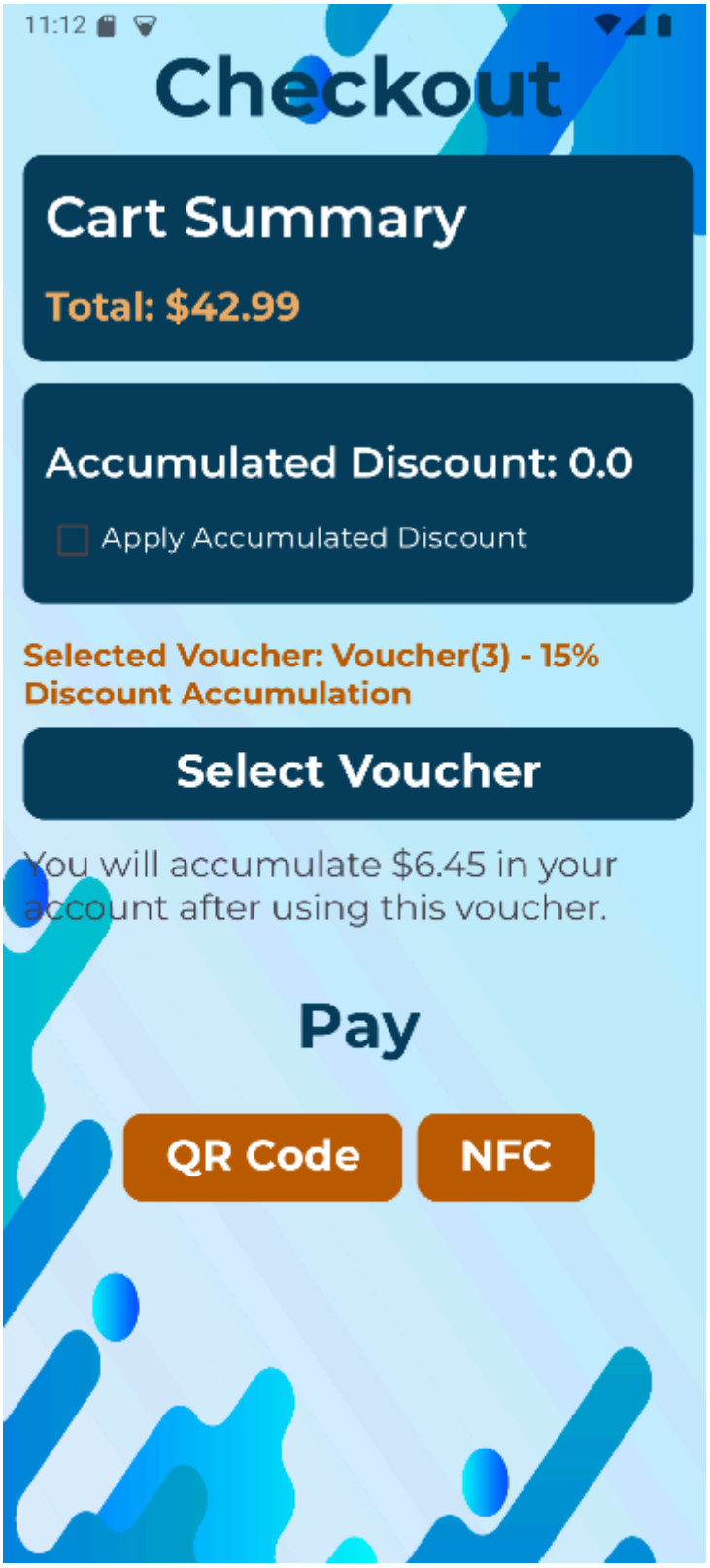
Now, by clicking on “Checkout” we will go to the Checkout page. Here we can select whether we want to apply the accumulated discount or if we want to apply a voucher; and select whether we want to pay with QRcode or NFC.



If we click on “Select Voucher” we go to the Pick Voucher Page, where we can select any voucher available:



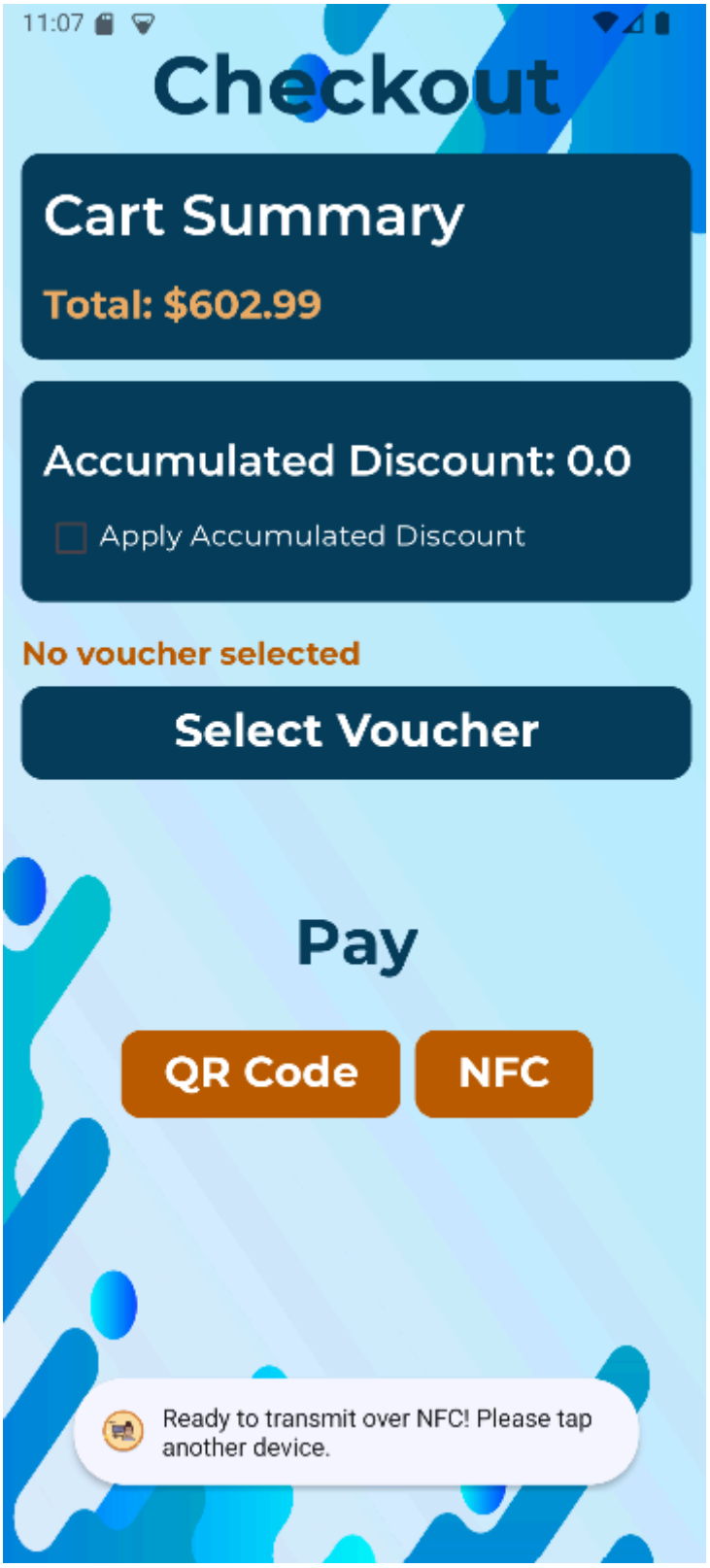
And the user can see it in Checkout Page:



If we click on the option “QR Code”, a Cart QR Code appears to be scanned by the terminal:



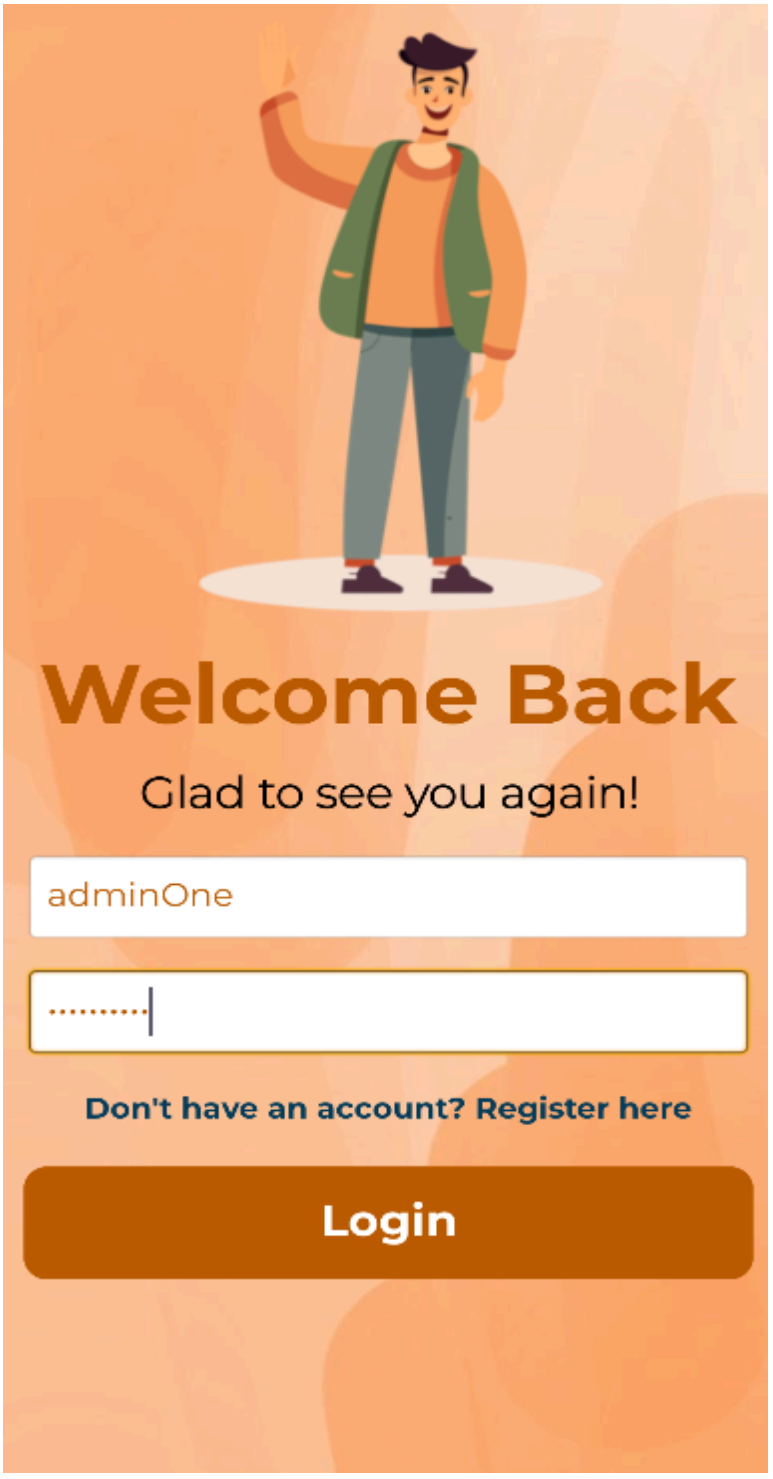
If we click on the option “NFC”, a message will appear to pay with NFC:



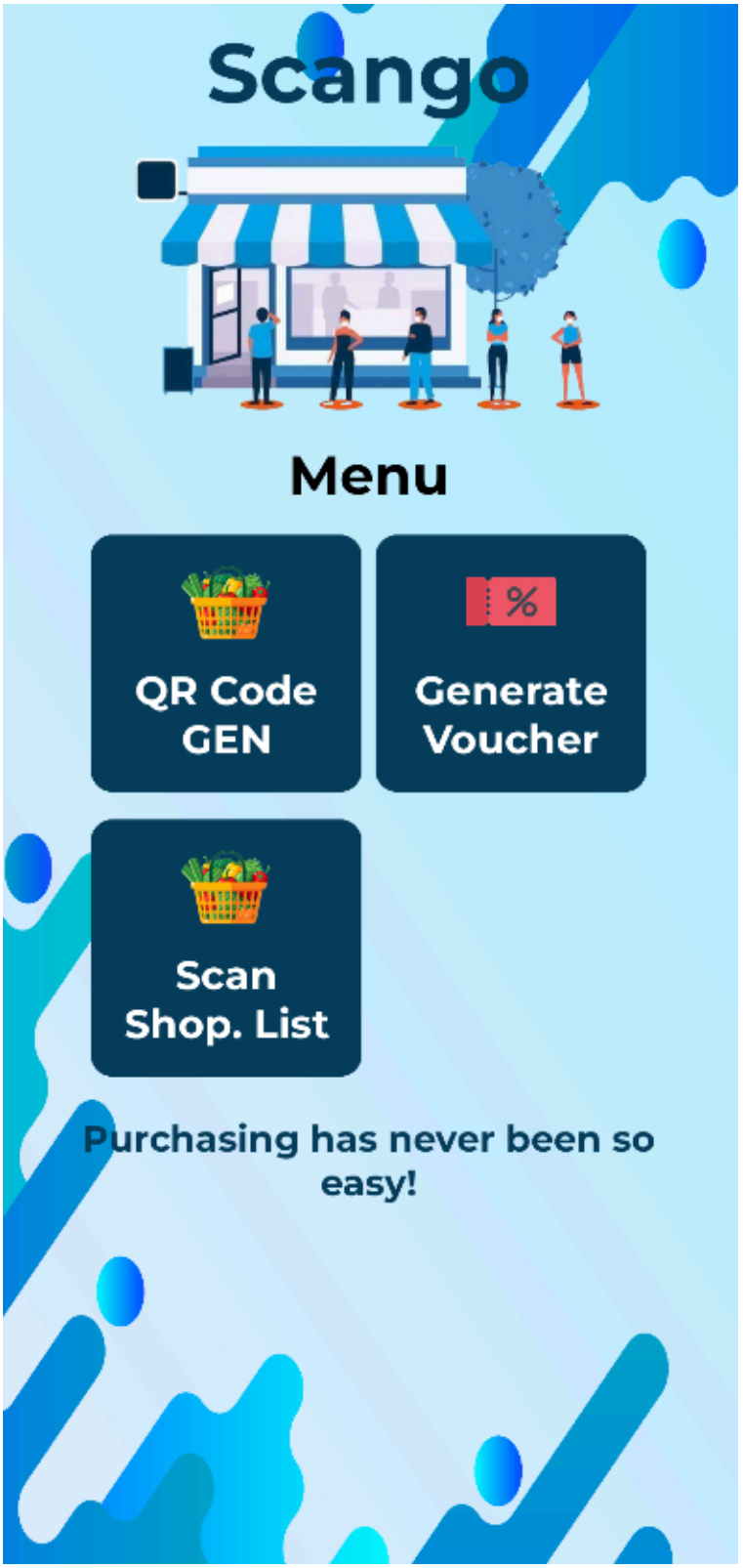
If we reopen the app, we are now redirected instead to the Login Page and not the Registration Page.

Field verifications are also applied to the Login Page, just like in the Registration Page.

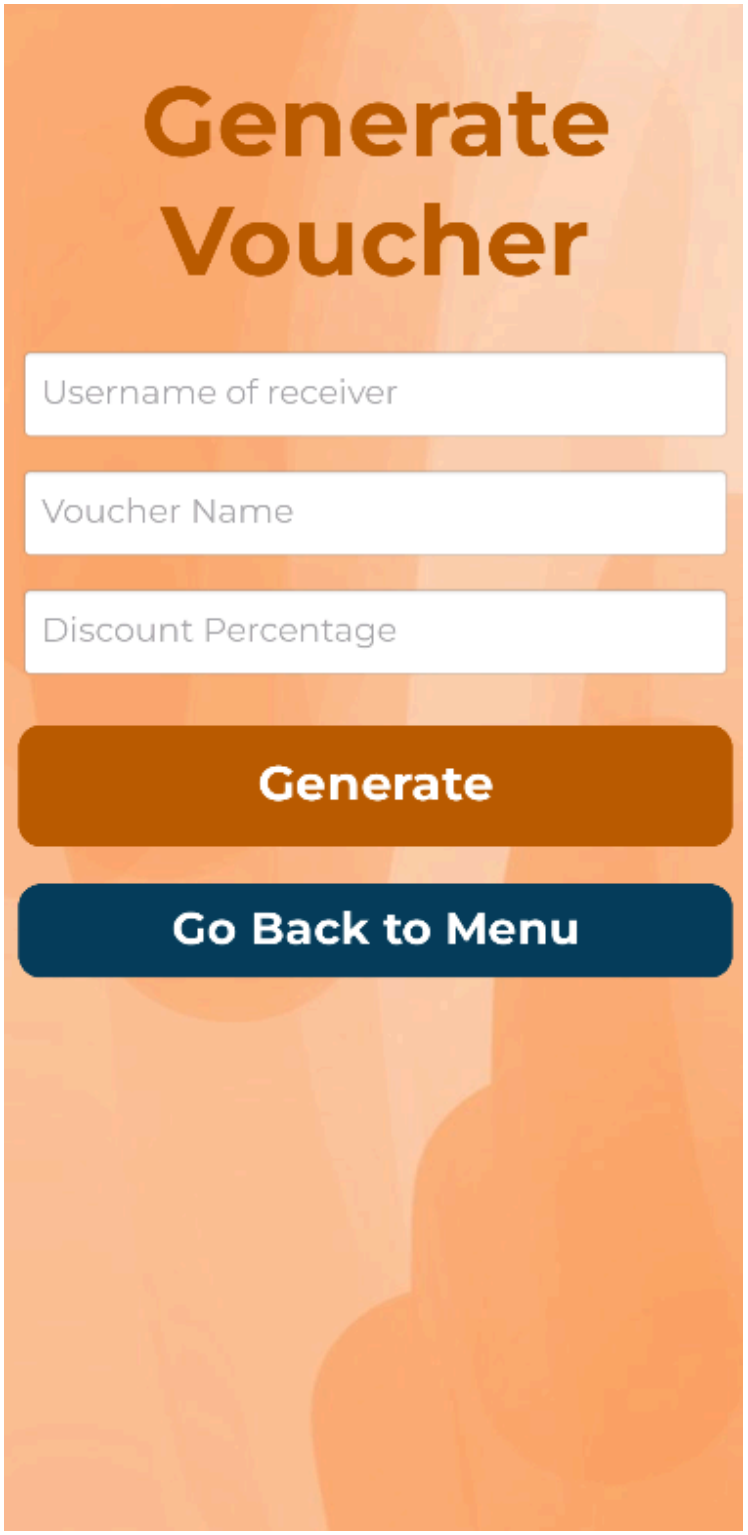
Now, we login as an admin to showcase the admin features we developed on our application:



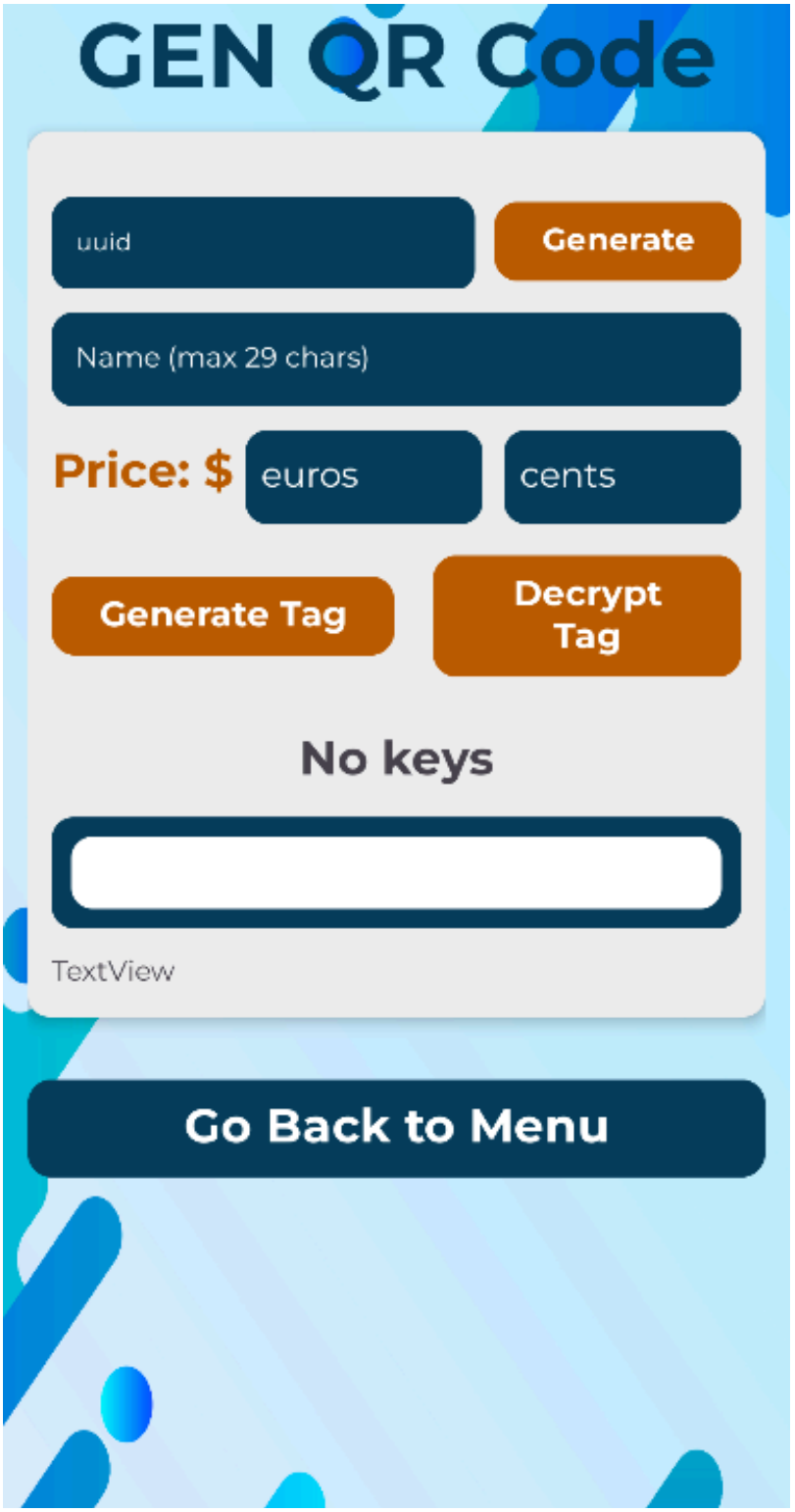
When logging in, we enter the home page again but this time with different options displayed:



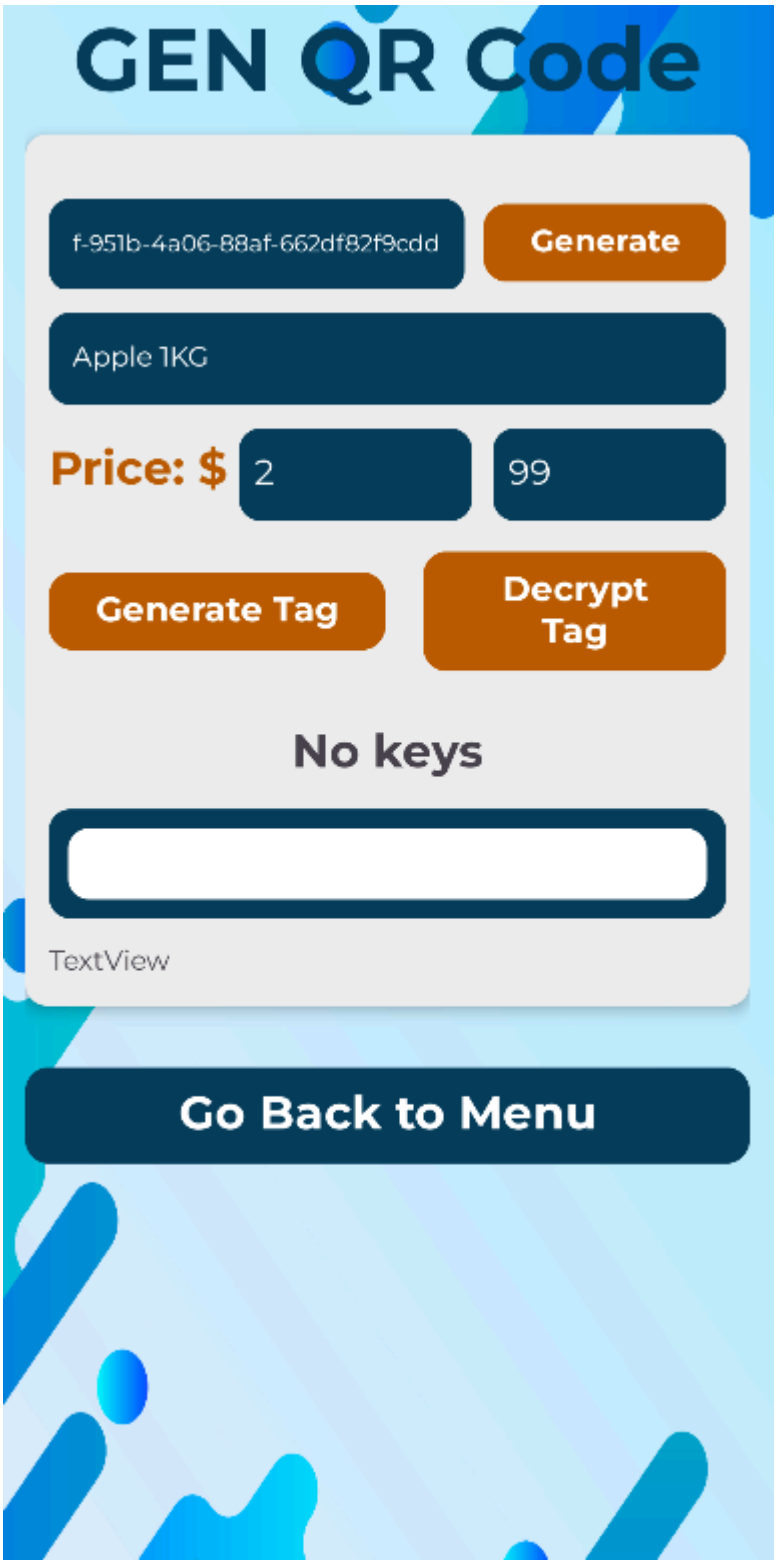
If we click on the “Generate Voucher” option, we are directed to a page responsible for generating vouchers for a specific user. We just need to insert the client’s username, the name of the voucher and the discount of the voucher:



After submitting the voucher, the client should now have the voucher in its account. Now, we head back to the Home Page and click on the “QR Code GEN” option. Here, we can generate a product’s QR Code for any product we want:



To generate a QR Code, we click on the “Generate” option to generate a random uuid for the product, insert the name of the product and the price in euros and cents:



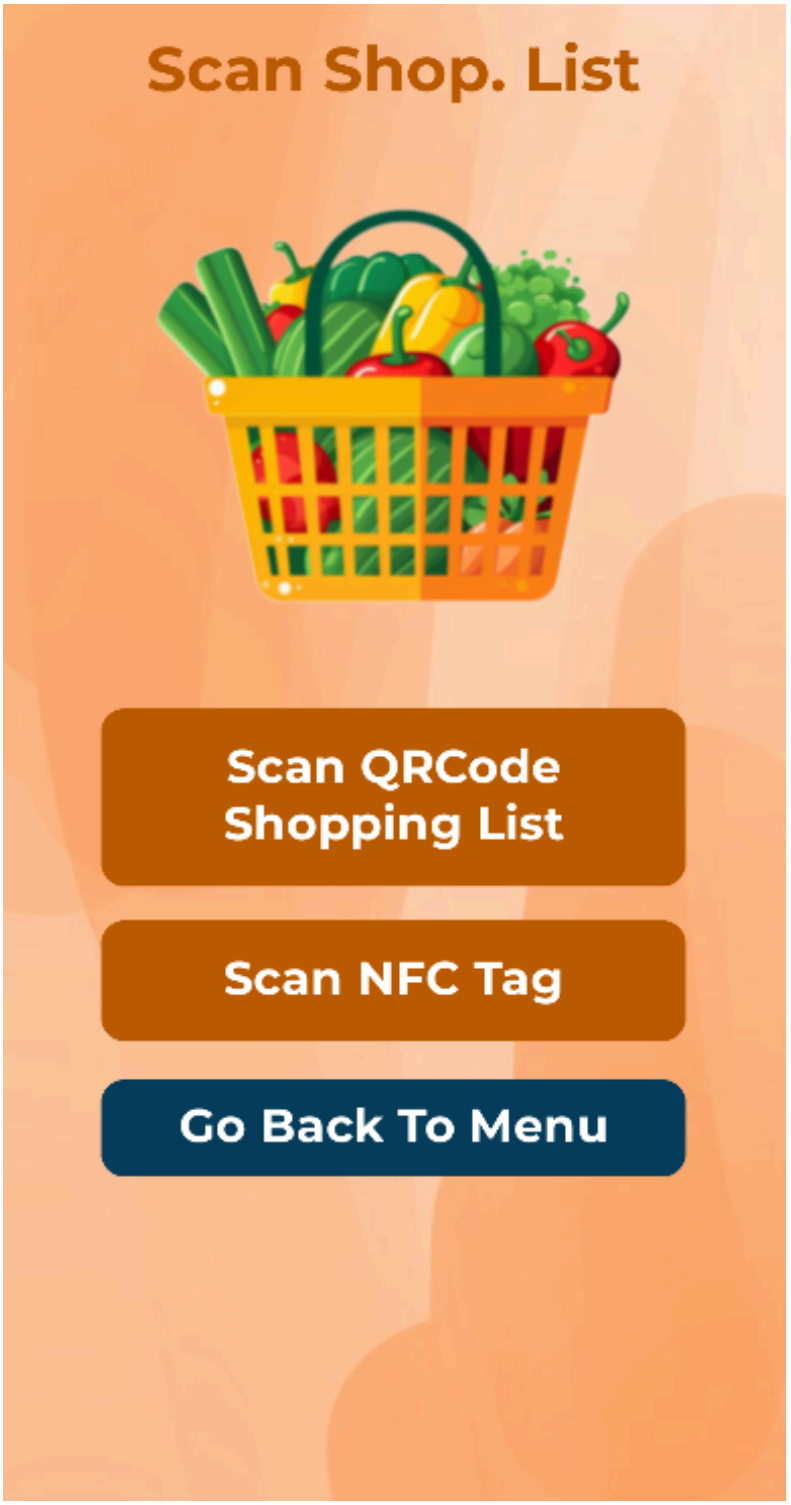
When clicking the “Encrypt Tag” option, the QR Code is generated and displayed:



After that, if we click the “Decrypt Tag” we can see the contents stored on the QR Code:



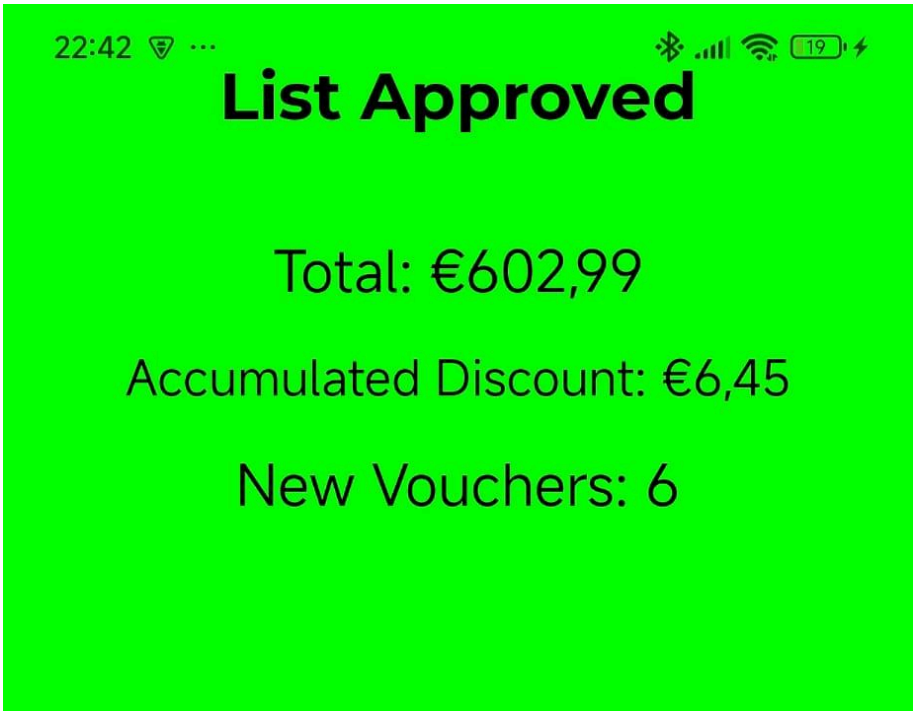
Heading back to the Home Page, we can click the final option called “Scan Shop. List”. Here, we can test the supermarket’s terminal for the checkout. Two options are available: checkout with QR Code or with NFC:



If we click on the “Scan QRCode Shopping List” option, we can scan the QRCode of the client.

If we click on the “Scan NFC Tag” option, we can scan the NFC tag of the client.

Finally, if the Shopping list is processed with success, the terminal will open the doors, show the total price, accumulated Discount and new vouchers added:



Or if it fails, the terminal will appear like this:

